
Πανεπιστήμιο Πατρών
Τμήμα Ηλεκτρολόγων Μηχανικών και Τεχνολογίας Υπολογιστών
Τομέας Συστημάτων και Αυτομάτου Ελέγχου

Προσομοίωση Δυναμικών Συστημάτων

Χαράλαμπος Μπεχλιούλης, Αναπληρωτής Καθηγητής
email: chmpechl@upatras.gr

Οκτώβριος 2021

1 Προσομοίωση Δυναμικών Συστημάτων	2
1.1 Εισαγωγή	2
1.2 Μέθοδος Runge-Kutta	4
1.3 Μέθοδος Πρόβλεψης-Διόρθωσης	6
1.4 Άκαμπα Συστήματα	8
1.5 Συναρτήσεις ode στο MATLAB	11
1.6 Επιλογή συνάρτησης	12
2 MATLAB-ODE Functions	14
2.1 Εισαγωγή	14
2.2 Συναρτήσεις ode	14
2.2.1 Βασική σύνταξη των συναρτήσεων ode	15
2.2.2 Εξίσωση Van der Pol	16
2.2.3 Η δομή options	17
2.3 Παραδείγματα	21
2.3.1 The Van der Pol equation - 1	22
2.3.2 The Van der Pol equation - 2	22
2.3.3 The Robertson problem	23
2.3.4 The bouncing ball problem	25

1.1 Εισαγωγή

Η προσομοίωση δυναμικών συστημάτων περιλαμβάνει την επίλυση διαφορικών εξισώσεων που είναι συνήθως στην μορφή προβλημάτων αρχικών συνθηκών. Οι αριθμητικές μέθοδοι που χρησιμοποιούνται για την επίλυση των διαφορικών εξισώσεων ονομάζονται αριθμητικοί ολοκληρωτές. Έχει εκπονηθεί ένας μεγάλος αριθμός εργασιών για τους αριθμητικούς ολοκληρωτές και μια ευρεία γκάμα μεθόδων είναι διαθέσιμη. Τα τελευταία χρόνια νέα και ισχυρά εργαλεία είναι διαθέσιμα γεγονός που καθιστά πολύ εύκολη την εκτέλεση προσομοιώσεων απ' ό,τι στο παρελθόν. Τέτοια εργαλεία είναι το **MATLAB**, το **SIMNON**, το **IMSL**, το **NAG** και το **SIMULINK**.

Στο κεφάλαιο αυτό παρουσιάζονται και αναλύονται διάφοροι αριθμητικοί ολοκληρωτές και γίνεται προσπάθεια να κατατοπιστεί ο αναγνώστης στο πώς να επιλέγει την κατάλληλη μέθοδο για τα προβλήματά του. Ένα δυναμικό σύστημα μπορεί να μοντελοποιηθεί στην μορφή του *Cauchy* ως εξής

$$\begin{aligned}\dot{x}_1 &= f_1(t, x) \\ &\vdots \\ \dot{x}_n &= f_n(t, x)\end{aligned}$$

ή χρησιμοποιώντας διανυσματικό συμβολισμό

$$\dot{x} = f(t, x)$$

Στην περιγραφή μας από εδώ και πέρα θα θεωρούμε την ειδική περίπτωση $n = 1$ και απλά θα γενικεύουμε τα αποτελέσματα. Η επίλυση μιας διαφορικής εξίσωσης σημαίνει, εξ' ορισμού,

την παραγωγή μιας συνάρτησης Φ , τέτοιας ώστε

$$\dot{\Phi} = f(t, \Phi)$$

Μπορεί να αποδειχθεί ότι, εάν η f πληρεί κάποιες ελάχιστες προϋποθέσεις κανονικότητας, για κάθε αρχική συνθήκη $x_0 = x(t_0)$ υπάρχει στην περιοχή του σημείου (t_0, x_0) μια μοναδική συνάρτηση $\Phi(t)$ η οποία επιλύει την εξίσωση και τέτοια, ώστε να είναι $\Phi(t_0) = x_0$ (επίσης θεωρούμε ότι έχει άπειρες συνεχείς παραγώγους). Το θεώρημα αυτό της ύπαρξης λύσης μας λέει ότι υπάρχει μια λύση αλλά δεν μας δίνει καμιά ένδειξη για τον τρόπο με τον οποίο θα την προσδιορίσουμε. Έτσι, μια αριθμητική επίλυση του προβλήματος του *Cauchy* είναι μια ακολουθία τιμών $[x_0, x_1, \dots, x_n]$ η οποία αντιστοιχεί σε μια αύξουσα ακολουθία χρονικών στιγμών $[t_0, t_1, \dots, t_n]$, τέτοια ώστε η τιμή x_i να προσεγγίζει την $\Phi(t_i)$. Τα σημεία t_i συχνά επιλέγονται από το χρήστη, αλλά είναι εν γένει προτιμότερο για λόγους συνέπειας να αφήνονται στη ρουτίνα να τα ορίσει. Εάν κάποιος χρειάζεται τη λύση σε ένα συγκεκριμένο σημείο, τότε συνήθως εφαρμόζεται κάποια μέθοδος παρεμβολής και βρίσκεται η λύση στο επιθυμητό σημείο.

Εφόσον δίνεται η αρχική τιμή x_0 κατά τη χρονική στιγμή t_0 , ο βασικός αλγόριθμος θα πρέπει να είναι ικανός να παράγει την τιμή $x_1 = x(t_1)$ κατά τη χρονική στιγμή $t_1 = t_0 + \Delta t$. Αντιλαμβανόμαστε βέβαια ότι η διαδικασία αυτή εισάγει κάποιο τοπικό σφάλμα, το οποίο στις περισσότερες περιπτώσεις μειώνεται εάν λάβουμε μικρότερα βήματα χρόνου Δt . Αυτό το τοπικό σφάλμα αποτελείται από δύο όρους, τον πρώτο που εξαρτάται από την αριθμητική ακρίβεια του επεξεργαστή, και το δεύτερο που εξαρτάται από τον ίδιο τον αλγόριθμο, ο οποίος, όπως θα δούμε παρακάτω, προσεγγίζει την διαφορική εξίσωση με μια αλγεβρική έκφραση. Μπορούμε να επαναλάβουμε τον αλγόριθμο, ώστε να παράγουμε από τη νέα αρχική συνθήκη x_1 που αναφέρεται στη χρονική στιγμή t_1 ένα νέο σημείο x_2 , που προσεγγίζει τη Φ για τη χρονική στιγμή $t_2 = t_1 + \Delta t$ (είτε το ίδιο Δt όπως προηγουμένως, οπότε ο αλγόριθμος ονομάζεται σταθερού βήματος, είτε διαφορετικό, οπότε ο αλγόριθμος ονομάζεται μεταβλητού βήματος). Με αυτόν τον τρόπο παράγουμε βήμα-βήμα μια αριθμητική λύση στο πρόβλημα του *Cauchy*. Αλλά όταν υπολογίζουμε την τιμή του x_2 , μαζί με το τοπικό σφάλμα εισάγουμε ένα επιπλέον σφάλμα, το οποίο δεν υπάρχει στο αρχικό βήμα: η ορθή αρχική συνθήκη για το διάστημα $[t_1, t_2]$ είναι η $\Phi(t_1)$, την οποία δε γνωρίζουμε, και γι' αυτό θέτουμε ως αρχική συνθήκη την τιμή x_1 , που αποτελεί προσέγγιση της $\Phi(t_1)$. Αυτό το σωρευτικό σφάλμα αυξάνει, εν γένει, όσο αυξάνεται ο αριθμός των βημάτων.

Ο πιο απλός στη σύλληψη τρόπος για να υπολογίσουμε το $x(t + \Delta t)$ είναι με το ανάπτυγμα *Taylor*. Σημειώστε ότι, εφόσον γνωρίζουμε την $\Phi'(t) = f(t, x)$ γνωρίζουμε εν δυνάμει όλες τις παραγώγους της λύσης. Για την ακρίβεια είναι

$$\begin{aligned}\Phi''(t) &= \frac{d}{dt}f(t, x) \\ &= \frac{\partial}{\partial t}f(t, x) + \frac{\partial f(t, x)}{\partial x} \frac{dx}{dt} = \frac{\partial}{\partial t}f(t, x) + \frac{\partial f(t, x)}{\partial x} f(t, x)\end{aligned}$$

και, λαμβάνοντας διαδοχικές παραγώγους της f , είναι εν γένει δυνατό να υπολογίσουμε όλες τις παραγώγους της Φ . Στην συνέχεια μπορούμε να προσδιορίσουμε την $\Phi(t_0 + \Delta t)$ χρησιμοποιώντας τους πρώτους n όρους του αναπτύγματος κατά *Taylor*. Κάνοντας αυτό συνεισφέρουμε στο τοπικό σφάλμα κατά ένα όρο που είναι ανάλογος της ποσότητας $(\Delta t)^{n+1}$. Αυτός ο όρος ονομάζεται σφάλμα αποκοπής, επειδή αποτελεί συνέπεια της λήψης ενός πεπερασμένου αριθμού όρων της σειράς *Taylor*, η οποία έχει άπειρους όρους. Για παράδειγμα, εάν λάβουμε υπόψη μόνο τους δύο πρώτους όρους της σειράς *Taylor* της f , λαμβάνουμε τον ακόλουθο τύπο

$$x_1 \approx \Phi(t_0 + \Delta t) \approx x_0 + \left[f + \frac{1}{2} \left(\frac{\partial f}{\partial t} + \frac{\partial f}{\partial x} f \right) \Delta t \right] \Delta t$$

ο οποίος ενέχει ένα σφάλμα της τάξης του $(\Delta t)^3$. Ο όρος που εμφανίζεται στις αγκύλες μπορεί να ερμηνευθεί ως η μέση κλίση της λύσης στο διάστημα $[t_0, t_0 + \Delta t]$ και θα αναφέρεται ως κλίση *Taylor*.

Η εφαρμογή αυτής της μεθόδου απαιτεί σε όλες τις πρακτικές περιπτώσεις, μια απίστευτα μεγάλη προκαταρκτική εργασία για τον προσδιορισμό των τύπων των διαδοχικών παραγώγων αλλά μέχρι το τέλος του 19^{ου} αιώνα δεν υπήρχε καλύτερος τρόπος για τον υπολογισμό της αριθμητικής λύσης μιας απλής διαφορικής εξίσωσης. Στο υπόλοιπο αυτού του κεφαλαίου θα παρουσιάσουμε εναλλακτικές μεθόδους αριθμητικής επίλυσης διαφορικών εξισώσεων και ειδικότερα τη μέθοδο *Runge-Kutta*, τη μέθοδο πρόβλεψης-διόρθωσης και ειδικές μεθόδους για άκαμπτες εξισώσεις.

1.2 Μέθοδος Runge-Kutta

Στην αρχή του 20^{ου} αιώνα οι Γερμανοί *Karl Runge* & *Wilhelm Kutta* ανέπτυξαν ανεξάρτητα μια οικογένεια αλγορίθμων, βασιζόμενοι στην παρατήρηση ότι, αντί να υπολογίζονται οι τιμές $f(t, x)$ και οι παράγωγοί τους στο σημείο t_0 , είναι αρκετό να προσδιοριστεί μόνο η κλίση σε διάφορα σημεία στο διάστημα $[t_0, t_0 + \Delta t]$ και να παραχθεί έτσι μια προσέγγιση $\Phi(t_0, t_0 + \Delta t)$ παρόμοια με αυτή που λαμβάνεται από το ανάπτυγμα κατά *Taylor*.

Για παράδειγμα, εάν υπολογιστεί η $f(t, x)$ για δύο διαφορετικές τιμές των t και x , είναι δυνατό να γραφεί ένας τύπος για την $x(t_0 + \Delta t)$ που να διαφέρει από την πραγματική λύση $\Phi(t_0 + \Delta t)$ κατά μια ποσότητα της τάξης του $(\Delta t)^3$, όπως συμβαίνει με την προσέγγιση δευτέρου βαθμού κατά *Taylor*. Σε κάθε βήμα πρέπει να υπολογίζονται οι ποσότητες

$$\begin{aligned} \kappa_1 &= f(t_0, x_0) \\ \kappa_2 &= f(t_0 + a\Delta t, x_0 + \alpha\kappa_1\Delta t) \end{aligned}$$

και επίσης η μέση κλίση για το διάστημα, η οποία είναι

$$\kappa = \lambda_1 \kappa_1 + \lambda_2 \kappa_2$$

Κατ' αναλογία με την κλίση *Taylor* που περιγράφηκε στην προηγούμενη ενότητα η παραπάνω κλίση κ θα ονομασθεί κλίση *Runge-Kutta*. Χρησιμοποιούμε την κλίση *Runge-Kutta* για να παράγουμε την τιμή $x(t_0 + \Delta t)$ ως εξής

$$x(t_0 + \Delta t) = x_0 + \kappa \Delta t$$

Εφόσον επιλέξουμε τις τιμές $\lambda_1, \lambda_2, \alpha$ έτσι ώστε να είναι

$$\begin{aligned} \lambda_1 + \lambda_2 &= 1 \\ \alpha \lambda_2 &= \frac{1}{2} \end{aligned}$$

η κλίση *Runge-Kutta* συμπίπτει, έως όρους ανώτερης τάξης, με την κλίση *Taylor*, όπως μπορεί να διαπιστωθεί εάν γραφεί το ανάπτυγμα κατά *Taylor* της κλίσης κ .

$$\begin{aligned} \kappa &= \lambda_1 \kappa_1 + \lambda_2 \kappa_2 \\ &= \lambda_1 f + \lambda_2 (f + \alpha \dot{f} \Delta t) \\ &= (\lambda_1 + \lambda_2) f + \alpha \lambda_2 \dot{f} \Delta t \\ &= f + \frac{1}{2} \dot{f} \Delta t \end{aligned}$$

Η μέθοδος μπορεί να γενικευτεί για ανώτερες τάξεις. Για παράδειγμα, είναι δυνατό να γράψουμε έναν αλγόριθμο *Runge-Kutta* τετάρτης τάξης που να απαιτεί τέσσερις φορές την κλίση της $f(t, x)$ σε κάθε βήμα και να παράγει αποτελέσματα ακριβή κατά ένα παράγοντα $(\Delta t)^5$, όπως θα έκανε και το ανάπτυγμα *Taylor* εάν κρατούνταν οι τέσσερις πρώτοι όροι.

Η μέθοδος των *Runge-Kutta* ήταν η πιο δημοφιλής μέθοδος αριθμητικής επίλυσης συνήθων διαφορικών εξισώσεων για περισσότερο από μισό αιώνα, κυρίως επειδή είναι εύκολο να χρησιμοποιείται με υπολογιστές χειρός, παλαιότερα, και να αξιοποιηθεί από τους ηλεκτρονικούς υπολογιστές, τις τελευταίες δεκαετίες. Το κύριο μειονέκτημά της είναι η αδυναμία προσδιορισμού της ακρίβειάς της. Η τυπική μέθοδος για τον παραπάνω προσδιορισμό ήταν να υπολογιστεί εκ νέου η λύση θεωρώντας διπλάσιο αριθμό σημείων και να θεωρείται αποδεκτό το αποτέλεσμα όταν οι δύο λύσεις ήταν ικανοποιητικά κοντά. Αυτό σήμαινε 200% αύξηση του υπολογιστικού χρόνου, και, εάν ο χρήστης αποδεικνυόταν άτυχος και τα αποτελέσματα δεν ήταν αρκετά κοντά, θα έπρεπε να επαναλάβει τους υπολογισμούς με μικρότερο, αυτή τη φορά, διάστημα Δt .

Στην δεκαετία του 1950 εξελίχθηκε μια νέα οικογένεια αλγορίθμων η οποία ονομάστηκε αλγόριθμοι 'πρόβλεψης-διόρθωσης'. Αυτοί οι αλγόριθμοι ήταν πιο αποτελεσματικοί από τους αλγορίθμους *Runge-Kutta* αντίστοιχης ακρίβειας και, ειδικότερα, με σύγκριση των αποτελεσμάτων της πρόβλεψης και της διόρθωσης μπορούσε να προσδιοριστεί το σφάλμα αποκοπής. Τελικά,

στη δεκαετία του 1970 προτάθηκε από τον *Fehlberg* μια τροποποίηση της μεθόδου Runge-Kutta. Όπως έχουμε δει, η προσέγγιση *Runge-Kutta* δεύτερης τάξης απαιτεί δύο προσδιορισμούς της συνάρτησης $f(t, x)$ ανά βήμα ολοκλήρωσης. Ο *Fehlberg* παρατήρησε ότι ένας επιπλέον προσδιορισμός, που εισάγει επιπλέον υπολογιστικό χρόνο 50%, επιτρέπει τον υπολογισμό του παράγοντα n που εμφανίζεται στον υπολογισμό του σφάλματος αποκοπής, το οποίο εκφράζεται με τη σχέση

$$\text{Σφάλμα} \approx n(\Delta t)^3 \quad (1.1)$$

Η κατάσταση βελτιώνεται για την προσέγγιση *Runge-Kutta* τετάρτης τάξης, η οποία απαιτεί σε κάθε βήμα τέσσερις προσδιορισμούς της $f(t, x)$. Τότε, ένας πέμπτος προσδιορισμός προκαλεί αύξηση μόλις 20% του υπολογιστικού χρόνου και παράγει μια προσέγγιση του σφάλματος αποκοπής. Σε κάθε βήμα ελέγχουμε το προσδιορισμένο τοπικό σφάλμα και θεωρούμε το βήμα ολοκλήρωσης ως επιτυχές, εάν το σφάλμα είναι μικρότερο από την τιμή

$$\max(RelTol\|x\|, AbsTol)$$

όπου *RelTol* και *AbsTol* είναι οι παράμετροι σχετικής και απόλυτης ανοχής, οι οποίες είναι σταθερές ποσότητες που μπορούν να οριστούν από το πρόγραμμα ή το χρήστη.

Η (1.1) δεν ικανοποιεί μονάχα τη δικαιολογημένη περιέργειά μας να γνωρίζουμε την τιμή του σφάλματος αποκοπής, αλλά επιπλέον μας επιτρέπει να κατασκευάζουμε πολύ αποτελεσματικούς αλγόριθμους μειώνοντας το βήμα ολοκλήρωσης, όταν το σφάλμα είναι μεγάλο και αυξάνοντάς το, όταν το σφάλμα είναι μικρό.

Μια τελευταία σημείωση που πρέπει να προσεχθεί: Ο αναγνώστης θα πρέπει να προσέχει να μην θέσει στις παραμέτρους *RelTol* και *AbsTol* υπερβολικά μικρές τιμές, επειδή ο αλγόριθμος θα αναγκαστεί να χρησιμοποιήσει πολύ μικρό βήμα ολοκλήρωσης και επομένως πολύ μεγάλο αριθμό βημάτων. Αυτό δε θα έχει επίπτωση μόνο στον απαιτούμενο υπολογιστικό χρόνο, αλλά θα οδηγήσει σε μεγάλα σφάλματα άλλου τύπου (πεπερασμένα αριθμητικά σφάλματα, σωρευτικά σφάλματα) τα οποία ο αλγόριθμος δεν λαμβάνει υπόψη.

Στην μέθοδο *Runge-Kutta* βασίζονται οι συναρτήσεις *ode23* και *ode45* που περιέχονται στο βασικό πακέτο του *MATLAB*.

1.3 Μέθοδος Πρόβλεψης-Διόρθωσης

Ας θεωρήσουμε ξανά το πρόβλημα του προσδιορισμού του $x_{k+1} \simeq \Phi(t_{n+1})$, αλλά θεωρώντας αυτήν τη φορά ότι γνωρίζουμε το ‘παρελθόν’ των μεταβλητών t και x , δηλαδή τις τιμές $[t_1, t_2, \dots, t_k]$ και $[x_1, x_2, \dots, x_k]$. Ας ορίσουμε επίσης την $\dot{x}_i = f(t_i, x_i)$, που είναι περίπου ίση

με τη $\dot{\Phi}(t_i)$. Τότε θα είναι

$$x_{k+1} = x_k + \int_{t_k}^{t_{k+1}} \dot{\Phi}(\tau) d\tau \quad (1.2)$$

Μπορούμε να επιλέξουμε να κάνουμε τον υπολογισμό του ολοκληρώματος με τη βοήθεια κάποιας αριθμητικής ρουτίνας βασισμένης, για παράδειγμα, στους κανόνες του τραπεζίου ή του *Simpson* ή σε κάποιον αλγόριθμο ανώτερης τάξης. Για να περιγράψουμε την ιδέα, ας επιλέξουμε την προσέγγιση με τον κανόνα του τραπεζίου. Τότε η παραπάνω εξίσωση θα γίνει

$$x_{k+1} \simeq x_k + \frac{\dot{x}_k + \dot{x}_{k+1}}{2} \Delta t = x_k + \frac{\dot{x}_k + f(t_{k+1}, x_{k+1})}{2} (t_{k+1} - t_k)$$

Επομένως η τιμή x_{k+1} μπορεί να ληφθεί λύνοντας την εξίσωση

$$x_{k+1} = x_k + \frac{\dot{x}_k + f(t_{k+1}, x_{k+1})}{2} \Delta t \quad (1.3)$$

Υπάρχουν διάφορες μέθοδοι για τον προσδιορισμό της τιμής του x_{k+1} που ικανοποιούν την παραπάνω πεπλεγμένη εξίσωση. Η ιδέα της πρόβλεψης - διόρθωσης είναι να αντικαταστήσουμε την τιμή x_{k+1} στο δεξιό μέλος της (1.3) με μια προβλεπόμενη τιμή $x_{k+1}^{(p)}$ που λαμβάνεται εάν αντικαταστήσουμε την ολοκληρωτέα ποσότητα της (1.2) με μια προσεγγιστική πολυωνυμική έκφραση. Για παράδειγμα, εάν επιλέξουμε να χρησιμοποιήσουμε την παραβολική προσέγγιση, θα πρέπει να χρησιμοποιήσουμε ένα πολυώνυμο δευτέρου βαθμού που περνά από τα σημεία $(t_{k-2}, \dot{x}_{k-2}), (t_{k-1}, \dot{x}_{k-1})$ και $(t_k, \dot{x}_k)$. Τώρα θα περιγράψουμε με πολύ απλούς όρους τη μέθοδο πρόβλεψης-διόρθωσης χρησιμοποιώντας γραμμική προσέγγιση. Έτσι το πολυώνυμό μας θα είναι απλά η γραμμή που περνά από τα σημεία $(t_{k-1}, \dot{x}_{k-1})$ και $(t_k, \dot{x}_k)$, δηλαδή το πολυώνυμο

$$P(\tau) = \dot{x}_k + \frac{\dot{x}_k - \dot{x}_{k-1}}{t_k - t_{k-1}} (\tau - t_k) \quad (1.4)$$

Μπορούμε να χρησιμοποιήσουμε αυτήν την τιμή στο δεξιό μέρος της (1.3) και να λάβουμε μια νέα ή, όπως λέγεται, 'διορθωμένη' τιμή για το x_{k+1} , η οποία θα είναι

$$\begin{aligned} x_{k+1} &= x_k + \frac{\dot{x}_k + \dot{x}_{k+1}}{2} (t_{k+1} - t_k) \\ &= x_k + \frac{\dot{x}_k + f^{(p)}(t_{k+1}, x_{k+1})}{2} (t_{k+1} - t_k) \end{aligned}$$

Η μέθοδος πρόβλεψης-διόρθωσης είναι εν γένει πολύ αποτελεσματική επειδή η σύγκριση ανάμεσα στην τιμή πρόβλεψης και την τιμή διόρθωσης οδηγεί σε ένα καλό προσδιορισμό του τοπικού σφάλματος που εισάγεται σε κάθε κύκλο ολοκλήρωσης, χωρίς να απαιτείται ένας επιπρόσθετος υπολογισμός όπως συμβαίνει στους αλγορίθμους *Runge-Kutta* και *Fehlberg*. Επιπρόσθετα, σε κάθε κύκλο μπορούμε να αλλάζουμε το βαθμό πρόβλεψης (γραμμικό, παραβολικό κ.α) πετυχαίνοντας βελτιωμένες συγκριτικά με τους άλλους αλγορίθμους λύσεις.

Το βασικό πακέτο του *MATLAB* περιλαμβάνει την συνάρτηση *ode113*, η οποία έχει μεταβλητό βήμα και χρησιμοποιεί αλγορίθμους πρόβλεψης-διόρθωσης μεταβλητού βαθμού (*Adams*) .

1.4 Άκαμπα Συστήματα

Στις προηγούμενες ενότητες, όπου είδαμε τη φιλοσοφία των μεθόδων *Runge-Kutta* και πρόβλεψης-διόρθωσης, αναλύσαμε το ζήτημα της ακρίβειας των λύσεων των αλγορίθμων αλλά δεν έγινε καμιά αναφορά για το πρόβλημα της ευστάθειας. Η ευστάθεια είναι μια ποιοτική ιδιότητα και είναι δύσκολο να οριστεί. Με τον όρο ακρίβεια αναφερόμαστε στην περίπτωση που ένας αλγόριθμος παράγει μια λύση μόλις λίγο διαφορετική από την πραγματική, και με τον όρο αστάθεια όταν ο αλγόριθμος παράγει μια λύση πολύ διαφορετική από την πραγματική. Με άλλα λόγια, μιλάμε για αστάθεια, όταν το σφάλμα γίνεται τόσο μεγάλο, ώστε η τιμή της λύσης να είναι πρακτικά άχρηστη για οποιοδήποτε σκοπό.

Έχειδειχθεί ότι κάθε αλγόριθμος *Runge-Kutta* δεύτερης τάξης γίνεται ασταθής μόλις το βήμα ολοκλήρωσης υπερβεί το διπλάσιο της ελάχιστης σταθεράς χρόνου του συστήματος. Οι αλγόριθμοι *Runge-Kutta* μεγαλύτερης τάξης μπορούν να δεχθούν μεγαλύτερα βήματα ολοκλήρωσης. Για παράδειγμα, οι αλγόριθμοι *Runge-Kutta* τετάρτης τάξης μπορούν να δεχθούν βήματα ολοκλήρωσης ίσα με το τετραπλάσιο της ελάχιστης σταθεράς χρόνου του συστήματος. Παρόμοια αποτελέσματα προκύπτουν στους αλγορίθμους πρόβλεψης-διόρθωσης.

Τώρα θα μελετήσουμε ένα σύστημα σαν το παρακάτω

$$\begin{aligned}\dot{x}_1 &= (a - 2)x_1 + 2(a - 1)x_2 \\ \dot{x}_2 &= (1 - a)x_1 + 2(1 - 2a)x_2\end{aligned}$$

το οποίο έχει τις εξής αρχικές συνθήκες:

$$\begin{aligned}x_1(0) &= 1 \\ x_2(0) &= 0\end{aligned}$$

Η λύση μπορεί πολύ εύκολα να υπολογιστεί αναλυτικά και είναι η ακόλουθη

$$\begin{aligned}x_1(t) &= 2e^{-t} - e^{-at} \\ x_2(t) &= -e^{-t} + e^{-at}\end{aligned}$$

δηλαδή είναι το άθροισμα δύο εκθετικών καμπύλων, μιας με σταθερά χρόνου $\tau_1 = 1$ και μιας άλλης με σταθερά χρόνου $\tau_2 = 1/a$. Όταν τα τ_1 και τ_2 είναι της ίδιας τάξης μεγέθους, το σύστημα μπορεί να επιλυθεί εύκολα με έναν αλγόριθμο σαν τον *ode23*. Η αριθμητική λύση είναι σωστή και ο υπολογιστικός χρόνος σε λογικά πλαίσια. Αν όμως τα τ_1 και τ_2 διαφέρουν κάποιες τάξεις μεγέθους ($\tau_1 = 1, \tau_2 = 1/2000$) τότε ο προσδιορισμός της λύσης απαιτεί πολύ

περισσότερο χρόνο. Ο όρος που ευθύνεται γι' αυτόν τον υπερβολικά μεγάλο αριθμό κύκλων ολοκλήρωσης και επομένως για τον απαιτούμενο υπολογιστικό χρόνο είναι ο e^{-at} , του οποίου η συνεισφορά στην τελική λύση, για τόσο μεγάλη τιμή του a , είναι σχεδόν ασήμαντη.

Ένα τέτοιο σύστημα ονομάζεται άκαμπτο. Είναι δύσκολο να δώσουμε έναν ακριβή ορισμό για την ακαμψία, καθώς αυτή είναι μια ποιοτική και όχι ποσοτική ιδιότητα. Τα άκαμπτα συστήματα δεν διακρίνονται πάντοτε εύκολα από τη μαθηματική τους διατύπωση. Οι μέθοδοι *Runge-Kutta* και πρόβλεψης-διόρθωσης δεν μπορούν να επιλύσουν ικανοποιητικά τέτοια άκαμπτα προβλήματα, καθώς τα βήματα ολοκλήρωσης καθορίζονται από την μικρότερη σταθερά χρόνου του συστήματος. Για να επιλύσουμε ένα άκαμπτο σύστημα χρειαζόμαστε έναν αλγόριθμο που να μπορεί να λαμβάνει βήματα ολοκλήρωσης πολύ μεγαλύτερα από την μικρότερη σταθερά χρόνου του συστήματος. Ο πρώτος αλγόριθμος τέτοιου είδους επινοήθηκε από τον *William Gear*. Η στρατηγική που ακολούθησε ήταν να επιλύσει την (1.3) με μια πεπλεγμένη μέθοδο, όπως τη μέθοδο *Newton-Raphson*.

Η λύση της εξίσωσης

$$\Phi(x) = 0$$

μπορεί συχνά να ληφθεί ξεκινώντας από μια ικανοποιητική αρχική τιμή και στην συνέχεια επιλύοντας επαναληπτικά την γραμμική εξίσωση

$$0 = \Phi(x_i) + \Phi'(x_i)(x_{i+1} - x_i)$$

Η μέθοδος διατηρεί την ισχύ της ακόμη και όταν το x δεν είναι βαθμωτό αλλά διανυσματικό μέγεθος με n συνιστώσες, $[x_1, x_2, \dots, x_n]$ και όταν η Φ δεν είναι μια εξίσωση αλλά ένα σύστημα εξισώσεων της μορφής

$$\Phi_1(x) = 0$$

$$\Phi_2(x) = 0$$

$$\vdots$$

$$\Phi_n(x) = 0$$

Ο ρόλος της παραγώγου σε αυτήν την περίπτωση πληρούται από την Ιακωβιανή ορίζουσα, που ορίζεται ως ο πίνακας με στοιχεία (i, j) της μορφής

$$\left(\frac{\partial \Phi}{\partial x}\right)_{i,j} = \frac{\partial \Phi_i}{\partial x_j}$$

Εάν αντικαταστήσουμε την τιμή x_{k+1} με την τιμή x , η εξίσωση που θέλουμε να επιλύσουμε

γίνεται

$$\Phi(x) = -x + x_k + \frac{\dot{x}_k + f(t_{k+1}, x)}{2}(t_{k+1} - t_k)$$

και η ιακωβιανή της Φ είναι η εξής

$$\frac{\partial \Phi}{\partial x} = -1 + \frac{t_{k+1} - t_k}{2} \frac{\partial}{\partial x} f(t_{k+1}, x)$$

Η μέθοδος *Newton-Raphson* μας κατευθύνει να επιλύσουμε την πεπλεγμένη εξίσωση με επαναληπτικό υπολογισμό των ποσοτήτων x_{i+1} , που είναι λύσεις του γραμμικού συστήματος

$$0 = \Phi(x_i) + \frac{\partial}{\partial x} \Phi(x_i)(x_{i+1} - x_i)$$

Η αρχική τιμή δεν αποτελεί πρόβλημα, καθώς πολλές ρουτίνες για άκαμπτα συστήματα χρησιμοποιούν την πρόβλεψη, για να παράγουν μια λογική αρχική τιμή, την ίδια που χρησιμοποιείται και στην μέθοδο πρόβλεψης-διόρθωσης. Επιπλέον, ο αριθμός των επαναλήψεων που απαιτούνται μεταβάλλεται από εφαρμογή σε εφαρμογή. Η επίλυση της γραμμικής εξίσωσης είναι μια διαδικασία που μπορεί στη σημερινή εποχή να πραγματοποιείται αρκετά αποδοτικά, και έτσι ο περισσότερος υπολογιστικός χρόνος αφιερώνεται στον υπολογισμό της ιακωβιανής. Είναι αλήθεια ότι οι άκαμπτοι αλγόριθμοι μας επιτρέπουν να λαμβάνουμε μεγάλα βήματα ολοκλήρωσης, αλλά εάν θα έπρεπε σε κάθε βήμα να υπολογίζουμε αριθμητικά την ιακωβιανή ορίζουσα και να επιλύουμε το σύστημα τότε ο υπολογισμός θα γινόταν πιθανώς πολύ αργός και θα έπρεπε να παραμείνουμε στην παλιά, γνωστή, μέθοδο των *Runge-Kutta*.

Στην πράξη οι καλές εκδοχές αυτής της στρατηγικής υπολογίζουν την ιακωβιανή ορίζουσα όχι σε κάθε βήμα αλλά μόνο όποτε χρειάζεται. Σε κάθε βήμα ο αλγόριθμος προσπαθεί να χρησιμοποιήσει την προηγούμενη ιακωβιανή και μόνο εάν η ακρίβεια προκύψει μικρότερη από κάποιο όριο επιχειρεί τον εκ νέου προσδιορισμό της ιακωβιανής.

Οι συναρτήσεις που περιλαμβάνονται στο βασικό πακέτο του *MATLAB* και επιλύουν άκαμπτα συστήματα είναι οι *ode23s*, *ode15s*, *ode23t* και *ode23tb*. Επιπρόσθετα, το *MATLAB* επιτρέπει να ενημερώνουμε τη χρονορουτίνα ότι

- η ιακωβιανή αποτελεί έναν αραιό πίνακα
- το σύστημα είναι γραμμικό, δηλαδή ότι η ιακωβιανή παραμένει σταθερή και δεν απαιτείται να προσδιορίζεται εκ νέου
- διαβιβάζουμε την ιακωβιανή εάν μπορούμε να την προσδιορίσουμε σε κλειστή μορφή

1.5 Συναρτήσεις ode στο MATLAB

Οι συναρτήσεις *ode* υλοποιούν μεθόδους αριθμητικής επίλυσης συνήθων διαφορικών εξισώσεων. Ξεκινώντας την αρχική χρονική στιγμή t_0 με την αρχική κατάσταση x_0 , προχωρούν υπολογίζοντας τη λύση για κάθε χρονικό βήμα. Αν η λύση που υπολογίζεται σε κάθε βήμα ικανοποιεί τα κριτήρια ανοχής σφάλματος που τίθενται τότε το βήμα θεωρείται επιτυχές. Σε αντίθετη περίπτωση θεωρείται αποτυχημένο και ο αλγόριθμος μειώνει το μέγεθος του χρονικού βήματος υπολογίζοντας εκ νέου τη λύση.

Υπάρχουν τρεις συναρτήσεις για μη-άκαμπτα προβλήματα

ode45 Βασίζεται στον άμεσο *Runge-Kutta* (4,5) τύπο (*Dormand-Prince*) . Στον υπολογισμό του $x(t_n)$ χρειάζεται μόνο τη λύση στο αμέσως προηγούμενο βήμα $x(t_{n-1})$. Γενικά, η *ode45* είναι η καλύτερη επιλογή για μια πρώτη δοκιμή στα περισσότερα προβλήματα.

ode23 Βασίζεται στον άμεσο *Runge-Kutta* (2,3) τύπο (*Bogacki-Shampine*) . Μπορεί να είναι πιο αποδοτική από την *ode45* σε χονδροειδείς ανοχές και στην παρουσία ήπιας ακαμψίας του προβλήματος. Όπως η *ode45* έτσι και η *ode23* χρειάζεται μόνο τη λύση στο αμέσως προηγούμενο βήμα $x(t_{n-1})$ για τον υπολογισμό του $x(t_n)$.

ode113 Αλγόριθμος μεταβλητής τάξης *Adams-Bashforth-Moulton PECE* . Μπορεί να είναι πιο αποδοτικός από τον *ode45* σε αυστηρές ανοχές και όταν ο υπολογισμός της συνάρτησης $\dot{x} = f(t, x)$ έχει μεγάλο υπολογιστικό κόστος. Ο αλγόριθμος χρειάζεται αρκετά παρελθοντικά σημεία για να υπολογίσει τη λύση.

Όλα τα δύσκολα προβλήματα δεν είναι άκαμπτα, αλλά όλα τα άκαμπτα προβλήματα είναι δύσκολα για αλγορίθμους που δεν είναι υλοποιημένοι γι' αυτά. Οι αλγόριθμοι για άκαμπτα συστήματα χρησιμοποιούνται ακριβώς όπως και οι προηγούμενοι αλγόριθμοι. Ωστόσο η αποδοτικότητά τους βελτιώνεται σημαντικά όταν τους παρέχονται επιπλέον πληροφορίες για το πρόβλημα.

Υπάρχουν τέσσερις συναρτήσεις για τα άκαμπτα προβλήματα

ode15s Αλγόριθμος μεταβλητής τάξης. Χρησιμοποιεί τύπους προς τα πίσω διαφορών (*Backward Differentiation Formulas* , *BDFs*) γνωστοί και ως *Gear* μέθοδοι. Χρησιμοποιεί όπως και η *ode113* αρκετά παρελθοντικά σημεία, για να υπολογίσει τη λύση. Είναι μια καλή επιλογή όταν υπάρχουν υπόνοιες για ακαμψία ή η *ode45* αποτυγχάνει ή είναι ιδιαίτερα αναποτελεσματική.

ode23s Βασίζεται σε έναν τροποποιημένο τύπο του *Rosenbrock* τάξης 2. Επειδή απαιτεί ένα παρελθοντικό σημείο για την επίλυση μπορεί να είναι πιο αποδοτική από την *ode15s* σε χονδροειδείς ανοχές. Μπορεί να λύσει ορισμένα είδη προβλημάτων τα οποία αποτυγχάνει να λύσει η *ode15s*.

ode23t Είναι μια υλοποίηση του κανόνα του τραπεζίου που χρησιμοποιεί μια ελεύθερη παρεμβολή. Η χρήση αυτού του αλγορίθμου είναι αποδοτική μόνο όταν έχουμε μέτρια άκαμπτο σύστημα και χρειαζόμαστε λύσεις χωρίς αριθμητικές απώλειες.

ode23tb Είναι μια υλοποίηση του *TR-BDF2*, ένας έμμεσος *Runge-Kutta* τύπος όπου το πρώτο στάδιο είναι ένα βήμα τραπεζοειδούς κανόνα και το δεύτερο στάδιο ένας προς τα πίσω διαφορών τύπος τάξης 2. Όπως και η *ode23s* μπορεί να είναι πιο αποδοτική από την *ode15s* σε χονδροειδείς ανοχές.

1.6 Επιλογή συνάρτησης

Δεν υπάρχει γενικός κανόνας που να ισχύει σε κάθε περίπτωση και γι' αυτό στην συνέχεια ακολουθούν προτάσεις που έχουν αποκτηθεί με την εμπειρία.

Η πρώτη και πιο σημαντική απόφαση είναι εάν θα πρέπει να χρησιμοποιήσετε μια άκαμπτη συνάρτηση ή μια μη-άκαμπτη. Δεν υπάρχουν σημαντικά πλεονεκτήματα εάν επιλέξετε μια άκαμπτη συνάρτηση, εκτός εάν το σύστημά σας έχει σταθερές χρόνου που διαφέρουν τουλάχιστον κατά δύο ή τρεις τάξεις μεγέθους. Οι άκαμπτες συναρτήσεις λαμβάνουν μεγάλα βήματα ολοκλήρωσης για να μην εισάγουν μεγάλα σφάλματα στην περιγραφή των μικρών σταθερών χρόνου.

Ο δεύτερος παράγοντας που θα πρέπει να ληφθεί υπόψη είναι ότι το κόστος κάθε πλήρους βήματος ολοκλήρωσης είναι συνήθως πολύ μεγάλο και γι' αυτό η επιλογή μιας άκαμπτης συνάρτησης θα πρέπει να γίνεται όταν δεν θα είναι αναγκαίος ο υπολογισμός της ιακωβιανής ορίζουσας σε κάθε βήμα. Οι συνέπειες της συνθήκης αυτής μπορούν να μετριαστούν, εάν υπολογιστεί η ιακωβιανή αναλυτικά και διαβιβαστεί στη συνάρτηση.

Γενικά μπορεί να λεχθεί ότι η μέθοδος πρόβλεψης-διόρθωσης είναι ανώτερη από τις μεθόδους *Runge-Kutta* όταν δεν υπάρχουν ασυνεχή φαινόμενα, δηλαδή όταν η παρεμβολή πέρα από τα όρια ενός πολυωνύμου οδηγεί σε καλή τιμή πρόβλεψης. Από την άλλη πλευρά, όταν η πρόβλεψη είναι παραπλανητική, τότε η μέθοδος πρόβλεψης-διόρθωσης υστερεί ως προς τις μεθόδους *Runge-Kutta*. Σ' αυτή την περίπτωση ανήκει το γνωστό πρόβλημα της μπάλας που αναπηδά επάνω σε ένα τραπέζι. Εδώ η προβλεπόμενη τροχιά της μπάλας διαπερνά το τραπέζι, κάτι που όμως δεν συμβαίνει. Ο αλγόριθμος του *MATLAB* που βασίζεται στην μέθοδο πρόβλεψης-διόρθωσης είναι ικανός να αντιμετωπίσει μια ασυνέχεια τέτοιας μορφής, αλλά σε βάρος της απόδοσης.

Εναλλακτικά, χρησιμοποιώντας ιδιότητες των συναρτήσεων ode μπορούμε να αντιμετωπίσουμε την ασυνέχεια που προκύπτει όταν η μπάλα κτυπά στο τραπέζι. Ένα τέτοιο φαινόμενο ονομάζεται στη γλώσσα του *MATLAB* γεγονός (**event**). Η ανίχνευση ενός τέτοιου γεγονότος (**event detection**) μπορεί να μοντελοποιηθεί ως ένα **zero-crossing** πρόβλημα ορίζοντας μια

συνάρτηση $g(x, t)$ τέτοια ώστε το γεγονός να συμβαίνει όταν ισχύει η συνθήκη

$$g(x, t) = 0$$

Το γεγονός μπορεί έτσι να ανιχνευθεί υπολογίζοντας την αριθμητική λύση x_n και ελέγχοντας για κάθε βήμα εάν υπάρχει εναλλαγή προσήμου από το $g(x_n, t_n)$ στο $g(x_{n+1}, t_{n+1})$. Εάν υπάρχει εναλλαγή προσήμου τότε βρίσκεται η χρονική στιγμή που συμβαίνει το γεγονός λύνοντας αριθμητικά για το a την εξίσωση

$$g(x_n(a), t_n + ah) = 0$$

Έτσι η χρονική στιγμή που συμβαίνει το γεγονός δίνεται ως $t_n + ah$.

Ένα άλλο είδος προβλημάτων που μπορούν να λυθούν με ορισμένες από τις συναρτήσεις του *MATLAB* είναι προβλήματα της μορφής

$$M(t, x)\dot{x} = f(t, x)$$

όπου ο πίνακας $M(t, x)$ ονομάζεται πίνακας μάζας (**mass matrix**). Όταν ο πίνακας $M(t, x)$ δεν είναι ιδιόμορφος τότε η πιο πάνω εξίσωση ισοδυναμεί με την εξίσωση

$$\dot{x} = M^{-1}(t, x)f(t, x)$$

και το πρόβλημα έχει λύση για κάθε αρχική κατάσταση x_0 . Όταν ο πίνακας $M(t, x)$ είναι ιδιόμορφος τότε το πρόβλημα $M(t, x)\dot{x} = f(t, x)$ είναι ένα πρόβλημα διαφορικών-αλγεβρικών εξισώσεων (**Differential-Algebraic Equation, DAE**). Το πρόβλημα αυτό έχει λύση όταν για την αρχική κατάσταση x_0 υπάρχει μια αρχική κλίση x_{p0} τέτοια ώστε να ισχύει $M(t, x)x_{p0} = f(t, x)$. Εάν δεν υπάρχει τέτοια αρχική κλίση τότε η συνάρτηση θεωρεί τα x_0, x_{p0} ως εκτιμήσεις και προσπαθεί να βρει τιμές κοντινές στις αρχικές για τις οποίες ισχύει η παραπάνω εξίσωση. Στην συνέχεια προχωρεί στην επίλυση της εξίσωσης όπως εάν το πρόβλημα ήταν συνήθων διαφορικών εξισώσεων. Το βασικό πακέτο του *MATLAB* περιλαμβάνει τις συναρτήσεις *ode15s* και *ode23t* που επιλύουν τέτοιου είδους προβλήματα.

Μια τελευταία παρατήρηση σχετικά με την τάξη ολοκλήρωσης: Οι αλγόριθμοι υψηλότερης τάξης, όπως ο *ode45*, είναι εν γένει αποδοτικότεροι από τους αλγορίθμους χαμηλότερης τάξης, όπως τον *ode23*, στην περιγραφή ομαλών συστημάτων. Οι αλγόριθμοι χαμηλότερης τάξης είναι μερικές φορές πιο αποτελεσματικοί, ειδικά όταν υπάρχουν ασυνέχειες ή όταν οι απαιτήσεις ακρίβειας είναι μικρότερες.

2.1 Εισαγωγή

Στο κεφάλαιο αυτό θα παρουσιάσουμε την αριθμητική επίλυση διαφορικών εξισώσεων χρησιμοποιώντας το MATLAB . Τα προβλήματα με τα οποία θα ασχοληθούμε είναι προβλήματα αρχικών συνθηκών (**Initial Value Problems, IVPs**) που περιγράφονται από συνήθεις διαφορικές εξισώσεις (**Ordinary Differential Equations, ODEs**) και διαφορικές-αλγεβρικές εξισώσεις (**Differential-Algebraic Equations, DAEs**) όπου η λύση που μας ενδιαφέρει ικανοποιεί ορισμένες συνθήκες την αρχική χρονική στιγμή.

2.2 Συναρτήσεις ode

Στον παρακάτω πίνακα δίνονται οι συναρτήσεις που διατίθενται στην βασική έκδοση του MATLAB . Επιπλέον, παρουσιάζεται το είδος του προβλήματος που επιλύει κάθε συνάρτηση καθώς και η μέθοδος που χρησιμοποιεί.

Ολοκληρωτής	Τύπος Προβλήματος	Μέθοδος
ode45	Μη άκαμπτες δ.ε.	<i>Runge-Kutta</i>
ode23	Μη άκαμπτες δ.ε.	<i>Runge-Kutta</i>
ode113	Μη άκαμπτες δ.ε.	<i>Adams</i>
ode15s	Άκαμπτες δ.ε. και DAEs	<i>NDFs(BDFs)</i>
ode23s	Άκαμπτες δ.ε.	<i>Rosenbrock</i>
ode23t	Μέτρια άκαμπτες δ.ε. και DAEs	<i>Trapezoidal Rule</i>
ode23tb	Άκαμπτες δ.ε.	<i>TR-BDF2</i>

Οι συναρτήσεις ode είναι σχεδιασμένες για να χειρίζονται συνήθεις διαφορικές εξισώσεις. Μια συνήθης διαφορική εξίσωση περιέχει μια ή περισσότερες παραγώγους της μεταβλητής x ως προς την ανεξάρτητη μεταβλητή t που είναι ο χρόνος. Η μεταβλητή x είναι συνήθως ένα διάνυσμα με στοιχεία $x_1, x_2, \dots, x_n$. Γενικά, υπάρχουν πολλές συναρτήσεις $x(t)$ που ικανοποιούν τη διαφορική εξίσωση οπότε χρειάζεται επιπρόσθετη πληροφορία για να προσδιορίσουμε τη λύση που μας ενδιαφέρει. Στα προβλήματα αρχικών συνθηκών η λύση που μας ενδιαφέρει ικανοποιεί συγκεκριμένες αρχικές συνθήκες x_0 τη χρονική στιγμή t_0 .

$$\begin{aligned}\dot{x} &= f(x, t) \\ x(t_0) &= x_0\end{aligned}$$

Αν η συνάρτηση $f(x, t)$ είναι ικανοποιητικά ομαλή, τότε το πρόβλημα έχει μια και μόνο μια λύση. Γενικά όμως δεν υπάρχει αναλυτική έκφραση για τη λύση, έτσι είναι απαραίτητο να προσεγγίσουμε τη λύση $x(t)$ με αριθμητικές μεθόδους όπως είναι οι ολοκληρωτές *ode*. Οι συναρτήσεις *ode* υλοποιούν μεθόδους αριθμητικής ολοκλήρωσης για την επίλυση των προβλημάτων αυτών. Ξεκινώντας την αρχική χρονική στιγμή t_0 με τις αρχικές συνθήκες x_0 , προχωρούν με χρονικά βήματα υπολογίζοντας τη λύση για κάθε βήμα. Αν αυτή η λύση ικανοποιεί τα κριτήρια ανοχής σφάλματος που τίθενται, τότε το βήμα θεωρείται επιτυχημένο. Αλλιώς, το βήμα είναι αποτυχημένο και ο ολοκληρωτής μικραίνει το βήμα και επιχειρεί νέο υπολογισμό.

2.2.1 Βασική σύνταξη των συναρτήσεων ode

Η πιο απλή σύνταξη, κοινή για όλες τις συναρτήσεις ode, είναι

```
[t, x]=ode_solver(@odefun, tspan, x0)
```

όπου `ode_solver` είναι μια από τις συναρτήσεις που προαναφέραμε.

Τα ορίσματα εισόδου είναι

odefun Η συνάρτηση που υπολογίζει το σύστημα των διαφορικών εξισώσεων. Έχει τη μορφή $dx=odefun(t, x)$ όπου t είναι βαθμωτό μέγεθος - χρόνος και dx, x είναι διανύσματα στήλες.

tspan Το διάστημα για το οποίο ο ολοκληρωτής επιλύει τη διαφορική εξίσωση. Για κάθε επιτυχημένο βήμα επιστρέφεται η υπολογισμένη λύση.

x_0 Το διάνυσμα αρχικών συνθηκών.

Τα ορίσματα εξόδου είναι

- t** Το διάνυσμα που περιέχει τις χρονικές στιγμές για τις οποίες ο ολοκληρωτής έχει υπολογίσει τη λύση της διαφορικής εξίσωσης.
- x** Ο πίνακας που περιέχει τη λύση της διαφορικής εξίσωσης για τις αντίστοιχες χρονικές στιγμές.

Στο παράδειγμα που ακολουθεί περιγράφεται η διαδικασία με την οποία λύνονται τα προβλήματα αρχικών συνθηκών για συνήθεις διαφορικές εξισώσεις. Η περιγραφή βασίζεται στην εξίσωση *Van der Pol*.

2.2.2 Εξίσωση *Van der Pol*

1. Γράφουμε το πρόβλημα στην μορφή συστήματος διαφορικών εξισώσεων πρώτης τάξης.

$$\dot{x} = f(x, t)$$

π.χ. μπορούμε να γράψουμε την εξίσωση *Van der Pol*

$$\ddot{x} - \mu (1 - x^2) \dot{x} + x = 0$$

ως ένα σύστημα δύο διαφορικών εξισώσεων πρώτης τάξης

$$\begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= \mu (1 - x_1^2) x_2 - x_1 \end{aligned}$$

2. Γράφουμε το σύστημα διαφορικών εξισώσεων σε κώδικα MATLAB. Η συνάρτηση πρέπει να έχει τη μορφή

```
function dx=odefun(t,x)
mi=1;
dx=[x(2);mi*(1-x(1)^2)*x(2)-x(1)];
```

3. Επιλύουμε τη διαφορική εξίσωση με έναν ολοκληρωτή. Αποφασίζουμε ποιόν ολοκληρωτή θα χρησιμοποιήσουμε και προσδιορίζουμε το διάστημα ολοκλήρωσης και τις αρχικές συνθήκες. Τέλος, καλούμε τον ολοκληρωτή για την επίλυση της διαφορικής εξίσωσης.

```
[t,x]=ode45(@odefun,[0 20],[2;0]);
```

Το διάνυσμα t περιέχει τις χρονικές στιγμές ολοκλήρωσης ενώ ο πίνακας x τη λύση σε αυτές τις χρονικές στιγμές.

4. Παρατηρούμε τη λύση της διαφορικής εξίσωσης. Χρησιμοποιώντας απλές εντολές plot μπορούμε να παραστήσουμε γραφικά την χρονική εξέλιξη των λύσεων.

2.2.3 Η δομή *options*

Για πιο σύνθετες εφαρμογές μπορούμε να ορίσουμε επιπρόσθετες παραμέτρους για το πρόβλημα μέσω της δομής *options*

options Μια δομή με προαιρετικές παραμέτρους που αλλάζουν τις ιδιότητες των μεθόδων ολοκλήρωσης. Η σύνταξη των συναρτήσεων ode που ενσωματώνει τη δομή *options* έχει ως εξής

```
[t, x] = ode_solver(odefun, tspan, x0, options);
```

Οι προεπιλεγμένες τιμές των παραμέτρων είναι τέτοιες ώστε να αντιμετωπίζουν τα περισσότερα προβλήματα. Σε ορισμένες περιπτώσεις όμως μπορούμε να βελτιώσουμε την απόδοση των ολοκληρωτών αλλάζοντας τις παραμέτρους αυτές.

Στον παρακάτω πίνακα δίνονται οι παράμετροι της δομής *options*.

Κατηγορία παραμέτρων	Παράμετροι
Έλεγχος σφάλματος	RelTol, AbsTol, NormControl
Έξοδος ολοκληρωτή	Refine, Stats
Ιακωβιανός πίνακας	Jacobian, JPattern
Πίνακας μαζας	Mass, MvPattern, MStateDependence
Ανίχνευση γεγονότος	Events

Η τροποποίηση των παραμέτρων αυτών γίνεται με τη συνάρτηση *odeset* που συντάσσεται ως εξής

```
options = odeset('name1', value1, 'name2', value2, ...)
```

Όσες από τις παραμέτρους δεν τροποποιούνται διατηρούν την προεπιλεγμένη τους τιμή.

Έλεγχος Σφάλματος

Σε κάθε βήμα ο ολοκληρωτής εκτιμάει το τοπικό σφάλμα $e(i)$ σε κάθε συνιστώσα της λύσης. Αυτό το σφάλμα πρέπει να είναι μικρότερο από το επιτρεπτό σφάλμα το οποίο είναι συνάρτηση

της καθορισμένης σχετικής ανοχής, *RelTol*, και της καθορισμένης απόλυτης ανοχής, *AbsTol*.

$$|e(i)| \leq \max \{RelTol|x(i)|, AbsTol(i)\} \quad (2.1)$$

Για τυπικά προβλήματα, οι ολοκληρωτές δίνουν ακρίβεια της τάξης που ζητάμε ενώ δίνουν μικρότερη ακρίβεια για προβλήματα που ο χρόνος ολοκλήρωσης είναι μεγάλος. Βέβαια τα δύσκολα προβλήματα απαιτούν και πιο αυστηρές ανοχές απ' ότι είναι οι προεπιλεγμένες.

Για να ρυθμίσουμε την σχετική ακρίβεια ελέγχουμε την παράμετρο *RelTol* ενώ για την απόλυτη ακρίβεια έχει μεγάλη σημασία το μέγεθος των λύσεων. Αν $|x(i)|$ είναι κάτι μικρότερο από το *AbsTol(i)* τότε ο αλγόριθμος δεν δίνει κανένα σωστό ψηφίο για το $x(i)$. Αυτό σημαίνει ότι θέλουμε *RelTol* ακριβή ψηφία στη λύση για τις συνιστώσες εκείνες βέβαια που είναι μεγαλύτερες από *AbsTol(i)*. Ακόμη και αν δεν ενδιαφερόμαστε για μια συνιστώσα $x(i)$ όταν είναι πολύ μικρή σε μέγεθος ίσως θα πρέπει να ορίσουμε το *AbsTol(i)* αρκούντως μικρό για να πάρουμε ορισμένα σωστά ψηφία στη λύση $x(i)$ έτσι ώστε να μπορούμε να υπολογίσουμε με ακρίβεια ορισμένες ποσότητες. Βέβαια για να διαπιστώσουμε το μέγεθος των λύσεων θα πρέπει να λύσουμε το πρόβλημα αρκετές φορές.

RelTol [Positive scalar $\{10^{-3}\}$] Η ανοχή σχετικού σφάλματος που εφαρμόζεται σε όλες τις συνιστώσες της λύσης x . Είναι ένα μέτρο του σφάλματος σχετικό με το μέγεθος της κάθε συνιστώσας της λύσης. Σε γενικές γραμμές, ελέγχει τον αριθμό των σωστών ψηφίων σε κάθε συνιστώσα της λύσης εκτός αυτών που είναι μικρότερες από *AbsTol(i)*.

AbsTol [Positive scalar or vector $\{10^{-6}\}$] Εκφράζει την ανοχή του απόλυτου σφάλματος σε κάθε συνιστώσα της λύσης. *AbsTol(i)* είναι ένα κατώφλι κάτω από το οποίο οι τιμές της λύσης δεν είναι σημαντικές, δηλαδή καθορίζει την ακρίβεια όταν η λύση προσεγγίζει το μηδέν. Αν το *AbsTol* δίνεται σαν διάνυσμα τότε θα πρέπει να έχει μήκος όσο και ο αριθμός των συνιστωσών της λύσης.

NormControl [on-{off}] Έλεγχος σφάλματος βασισμένος στην νόρμα της λύσης. Αν θέλουμε επιτυχημένο βήμα να είναι αυτό για το οποίο ισχύει $|e| \leq \max\{RelTol|x|, AbsTol\}$ τότε θέτουμε την παράμετρο αυτή on. Η προεπιλεγμένη τιμή είναι off και αναφέρεται στον έλεγχο 2.1

Έξοδος ολοκληρωτή

Οι παράμετροι αυτές επιτρέπουν στο χρήστη να ελέγχει την έξοδο των ολοκληρωτών.

Refine [Positive Integer] Αυξάνει το πλήθος των σημείων της λύσης κατά ένα παράγοντα της τιμής του *Refine*. Αν *Refine*=1 τότε ο ολοκληρωτής επιστρέφει τις τιμές όπως τις έχει υπολογίσει σε κάθε βήμα. Αν *Refine*>1 τότε ο ολοκληρωτής υποδιαιρεί το διάστημα

του βήματος ολοκλήρωσης σε Refine τον αριθμό υποδιαστήματα και επιστρέφει τιμή για κάθε σημείο. Η προκαθορισμένη τιμή Refine για όλες τις συναρτήσεις ode εκτός της ode45 είναι 1 ενώ για την ode45 είναι 4. Οι επιπλέον τιμές που παράγονται με το Refine υπολογίζονται βάσει τύπων παρεμβολής, εξειδικευμένων να χρησιμοποιούνται στις ode και να δίνουν πολύ καλή ακρίβεια χωρίς να αυξάνουν σημαντικά τον υπολογιστικό χρόνο.

Stats [on-[off]] Καθορίζει αν ο ολοκληρωτής θα δείξει στο τέλος της επίλυσης του προβλήματος ορισμένα στατιστικά στοιχεία για τους υπολογισμούς που έγιναν. Αν τεθεί on τότε μετά την επίλυση του προβλήματος δίνονται

- Ο αριθμός των επιτυχημένων βημάτων.
- Ο αριθμός των αποτυχημένων προσπαθειών.
- Ο αριθμός των υπολογισμών της $f(t, x)$.
- Ο αριθμός των υπολογισμών του πίνακα $\frac{\partial f}{\partial x}$.
- Ο αριθμός των LU αποσυνθέσεων.
- Ο αριθμός των γραμμικών συστημάτων που επιλύθηκαν.

Ιακωβιανός πίνακας

Οι ολοκληρωτές για άκαμπτα συστήματα δουλεύουν πιο γρήγορα και πιο αξιόπιστα αν τους παρασχεθεί επιπλέον πληροφορία για τον ιακωβιανό πίνακα $\frac{\partial f}{\partial x}$, τον πίνακα των μερικών παραγώγων της συνάρτησης $f(t, x)$.

$$\frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}$$

Ο ιακωβιανός πίνακας εφαρμόζεται μόνο στους ολοκληρωτές που λύνουν άκαμπτα συστήματα (ode15s, ode23s, ode23t, ode23tb) για τα οποία ο ιακωβιανός πίνακας μπορεί να είναι κρίσιμος για την αξιοπιστία και την αποδοτικότητα των ολοκληρωτών. Αν δεν παρασχεθεί ο ιακωβιανός πίνακας, οι ολοκληρωτές τον προσεγγίζουν αριθμητικά χρησιμοποιώντας πεπερασμένες διαφορές.

Jacobian [Function-Constant matrix] Ένας σταθερός πίνακας ή μια συνάρτηση που υπολογίζει τον ιακωβιανό πίνακα. Δίνοντας σε αναλυτική μορφή τον ιακωβιανό πίνακα αυξάνεται η ταχύτητα και η αξιοπιστία της λύσης για άκαμπτα συστήματα.

JPattern [Sparse matrix of {0,1}] Αραιός πίνακας με μονάδες εκεί όπου ο ιακωβιανός πίνακας είναι μη μηδενικός. Χρησιμοποιείται όταν δε δίνεται αναλυτικά ο ιακωβιανός πίνακας αλλά προσεγγίζεται αριθμητικά. Τα στοιχεία του πίνακα αυτού είναι $JP(i, j) = 1$ όταν το $f_i(t, x)$ εξαρτάται από το x_j και $JP(i, j) = 0$ αλλιώς. Ο ολοκληρωτής χρησιμοποιεί τον

πίνακα αυτό για να παράγει αριθμητικά τον ιακωβιανό πίνακα. Αν ο ιακωβιανός πίνακας είναι μεγάλος και αραιός τότε μπορεί να επιταχυνθεί πάρα πολύ η επίλυση των άκαμπτων προβλημάτων.

Πίνακας μάζας και DAEs

Ένα άλλο είδος προβλημάτων που μπορούν να λυθούν με τις χρονορουτίνες του *MATLAB* είναι προβλήματα της μορφής

$$M(t, x)\dot{x} = f(t, x) \quad (2.2)$$

όπου ο πίνακας $M(t, x)$ ονομάζεται πίνακας μάζας (**mass matrix**). Όταν ο πίνακας $M(t, x)$ δεν είναι ιδιόμορφος τότε η πιο πάνω εξίσωση ισοδυναμεί με την εξίσωση

$$\dot{x} = M^{-1}(t, x)f(t, x)$$

και το πρόβλημα έχει λύση για κάθε αρχική κατάσταση x_0 . Βέβαια η επίλυση της εξίσωσης 2.2 απευθείας είναι πιο βολική, ειδικά αν ο πίνακας M είναι αραιός, και μειώνει τις απαιτήσεις μνήμης και χρόνου επίλυσης του προβλήματος.

Όταν ο πίνακας $M(t, x)$ είναι ιδιόμορφος τότε το πρόβλημα $M(t, x)\dot{x} = f(t, x)$ είναι ένα πρόβλημα διαφορικών-αλγεβρικών εξισώσεων (**Differential-Algebraic Equation, DAE**). Το πρόβλημα αυτό έχει λύση όταν για την αρχική κατάσταση x_0 υπάρχει μια αρχική κλίση xp_0 τέτοια ώστε να ισχύει $M(t_0, x_0)xp_0 = f(t_0, x_0)$. Εάν δεν υπάρχει τέτοια αρχική κλίση τότε η χρονορουτίνα θεωρεί τα x_0, xp_0 ως εκτιμήσεις και προσπαθεί να βρει τιμές κοντινές στις αρχικές για τις οποίες ισχύει η παραπάνω εξίσωση. Στην συνέχεια προχωρεί στην επίλυση της εξίσωσης όπως εάν το πρόβλημα ήταν συνήθων διαφορικών εξισώσεων. Το βασικό πακέτο του *MATLAB* περιλαμβάνει τις χρονορουτίνες *ode15s* και *ode23t* που επιλύουν τέτοιου είδους προβλήματα.

Mass [Constant matrix-Function] Σταθερός πίνακας ή συνάρτηση που υπολογίζει τον πίνακα μάζας.

MvPattern [Sparse matrix] Πίνακας με μονάδες και μηδενικά. $MP(i, j) = 1$ αν για οποιοδήποτε k το (i, k) στοιχείο του $M(t, x)$ εξαρτάται από το j στοιχείο του x αλλιώς $MP(i, j) = 0$.

MStateDependence [none-strong] Εξάρτηση του πίνακα μάζας από τις καταστάσεις x . Αν ο πίνακας είναι του τύπου $M(t)$ τότε θέτουμε την παράμετρο αυτή none αλλιώς αν ο πίνακας είναι του τύπου $M(t, x)$ θέτουμε της παράμετρο strong.

Ανίχνευση Γεγονότος

Σε ορισμένα προβλήματα είναι σημαντική η χρονική στιγμή στην οποία συμβαίνει ένα γεγονός όπως στο παράδειγμα που η μπάλα χτυπάει στο έδαφος. Ενώ λοιπόν ο ολοκληρωτής

επιλύει το πρόβλημα μπορεί ταυτόχρονα να ανιχνεύει τέτοια γεγονότα προσδιορίζοντας μεταβάσεις από ή προς το μηδέν δεδομένων ποσοτήτων. Η ανίχνευση ενός τέτοιου γεγονότος (**event detection**) μπορεί να μοντελοποιηθεί ως ένα **zero-crossing** πρόβλημα ορίζοντας μια συνάρτηση $g(x, t)$ τέτοια ώστε το γεγονός να συμβαίνει όταν ισχύει η συνθήκη

$$g(x, t) = 0$$

Το γεγονός μπορεί έτσι να ανιχνευθεί υπολογίζοντας την αριθμητική λύση x_n και ελέγχοντας για κάθε βήμα εάν υπάρχει εναλλαγή προσήμου από το $g(x_n, t_n)$ στο $g(x_{n+1}, t_{n+1})$. Εάν υπάρχει εναλλαγή προσήμου τότε βρίσκεται η χρονική στιγμή που συμβαίνει το γεγονός λύνοντας αριθμητικά για το a την εξίσωση

$$g(x_n(a), t_n + ah) = 0$$

Έτσι η χρονική στιγμή που συμβαίνει το γεγονός δίνεται ως $t_n + ah$.

Events [Function] Συνάρτηση που περιέχει ένα ή περισσότερα γεγονότα. Είναι της μορφής

```
[value, isterminal, direction]=events(t, x)
```

όπου `value`, `isterminal` και `direction` είναι διανύσματα των οποίων το i στοιχείο αντιστοιχεί στο i γεγονός.

- `value(i)` η τιμή του i γεγονότος.
- `isterminal(i)=1` αν η επίλυση σταματά με την ανίχνευση του i γεγονότος ή `isterminal(i)=0` αν δεν σταματά.
- `direction(i)=1` αν το γεγονός συμβαίνει μόνο όταν το `value(i)` γίνεται μηδέν από τα αρνητικά (αυξάνεται), `direction(i)=-1` αν το γεγονός συμβαίνει μόνο όταν το `value(i)` γίνεται μηδέν από τα θετικά (μειώνεται) και `direction(i)=0` όταν το γεγονός συμβαίνει και στις δύο περιπτώσεις.

2.3 Παραδείγματα

Στην ενότητα αυτή παρουσιάζονται τέσσερα παραδείγματα που περιγράφουν τον προγραμματισμό των συναρτήσεων ode για την επίλυση προβλημάτων αρχικών συνθηκών. Μαζί με τους κώδικες παρουσιάζονται και τα αποτελέσματα των προσομοιώσεων διευκολύνοντας έτσι την κατανόηση των προγραμμάτων.

2.3.1 The Van der Pol equation - 1

Το παράδειγμα αυτό περιγράφει την επίλυση της διαφορικής εξίσωσης Van der Pol όταν δεν είναι άκαμπτη.

```
function Van_der_Pol

global mi

mi=1;

tspan=[0 20];
x0=[2;0];
[t,x]=ode45(@vdp,tspan,x0);

plot(t,x(:,1),'b-',t,x(:,2),'g-.')
title(['Solution of Van der Pol equation, mi = ',num2str(mi)])
xlabel('t(sec)')
ylabel('solution')
legend('x_1','x_2')
legend boxoff
grid
function dx=vdp(t,x)
global mi
dx=[x(2);mi*(1-x(1)^2)*x(2)-x(1)];
```

2.3.2 The Van der Pol equation - 2

Το παράδειγμα αυτό περιγράφει την επίλυση της διαφορικής εξίσωσης Van der Pol όταν είναι άκαμπτη.

```
function Van_der_Pol

global mi

mi=1000;

options=odeset('Jacobian',@J);
tspan=[0 2000];
x0=[2;0];
[t,x]=ode15s(@vdp,tspan,x0,options);
```

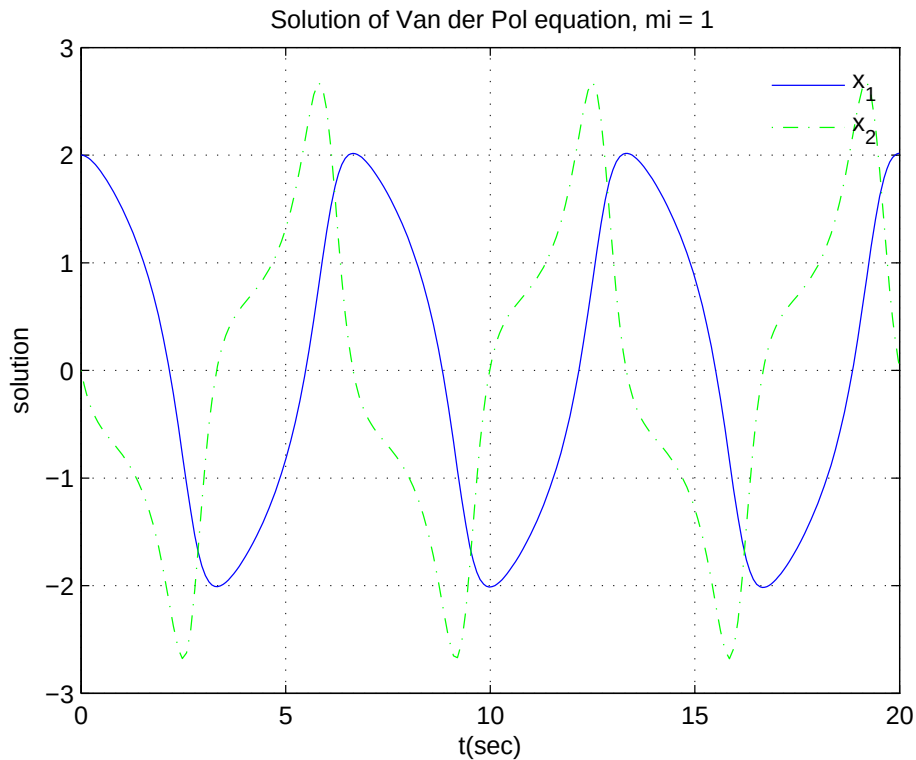


Figure 2.1: Μη άκαμπτη διαφορική εξίσωση Van der Pol

```

plot(t,x(:,1),'b-')
title(['Solution of Van der Pol equation, mi =',num2str(mi)])
xlabel('t(sec)')
ylabel('solution x_1')
legend boxoff
grid

function dx=vdp(t,x)
global mi
dx=[x(2);mi*(1-x(1)^2)*x(2)-x(1)];

function dfdx=J(t,x)
global mi
dfdx=[0 1;-2*mi*x(1)*x(2)-1 mi*(1-x(1)^2)];

```

2.3.3 The Robertson problem

Το παράδειγμα αυτό περιγράφει την επίλυση ενός διαφορικού αλγεβρικού προβλήματος.

```
function Robertson
```

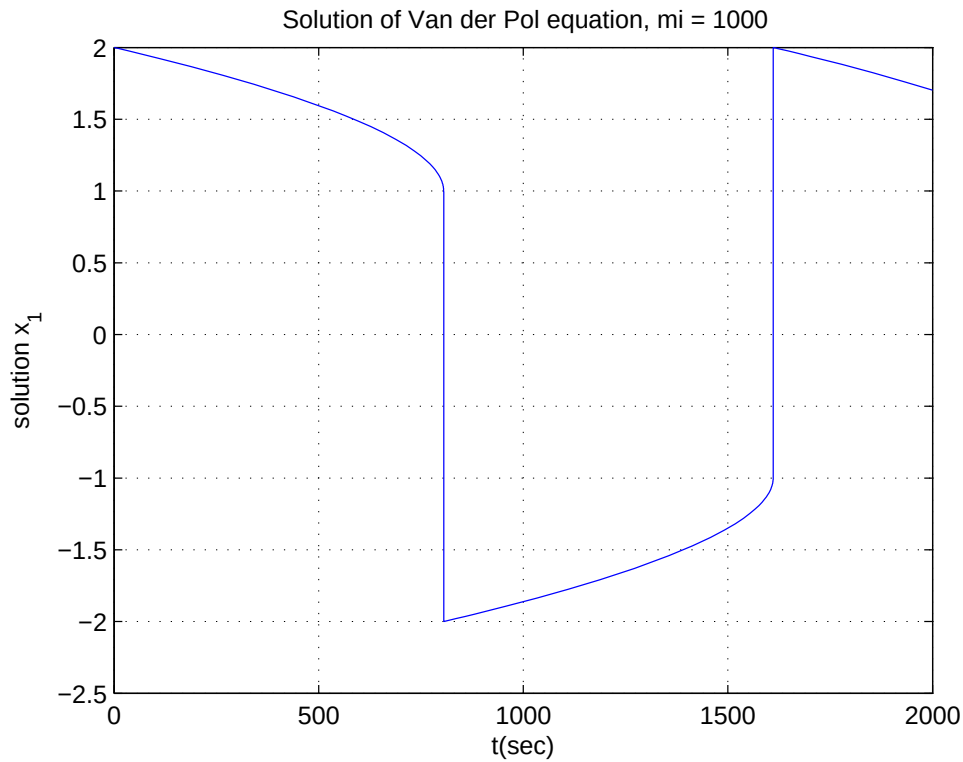


Figure 2.2: Άκαμπτη διαφορική εξίσωση Van der Pol

```
M=[1 0 0;0 1 0;0 0 0];
```

```
options=odeset('Mass',M,'RelTol',1e-4,'AbsTol',[1e-6 1e-10 1e-6]);
```

```
tspan=[0 4*logspace(-6,6)];
```

```
x0=[1;0;1e-3];
```

```
[t,x]=ode15s(@f,tspan,x0,options);
```

```
semilogx(t,[x(:,1) 1e4*x(:,2) x(:,3)]);
```

```
title('Robertson DAE Problem')
```

```
legend('x_1','1e4*x_2','x_3')
```

```
function dx=f(t,x)
```

```
dx=[-0.04*x(1)+1e4*x(2)*x(3);
```

```
0.04*x(1)-1e4*x(2)*x(3)-3e7*x(2)^2;
```

```
x(1)+x(2)+x(3)-1];
```

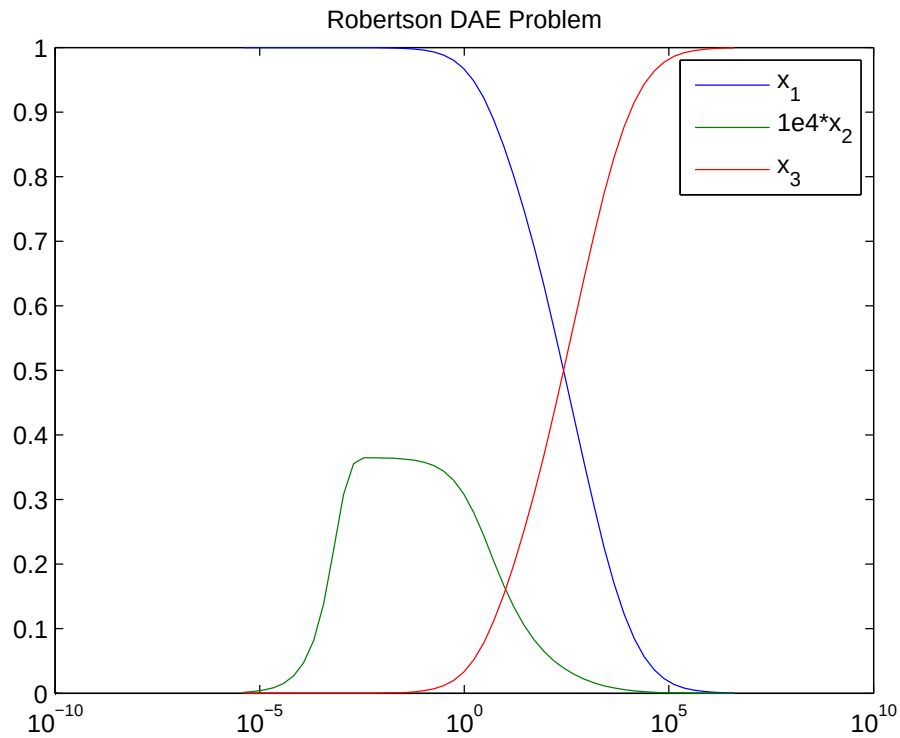


Figure 2.3: Διαφορική αλγεβρική εξίσωση Robertson

2.3.4 The bouncing ball problem

Το παράδειγμα περιγράφει την επίλυση ενός προβλήματος που περιλαμβάνει ανίχνευση γεγονότος.

```
function bouncing_ball

global g

g=10;
refine=12;

options=odeset('Events',@events,'Refine',refine);
tstart=0;
tfinal=20;
x0=[0;20];

tout=[];
xout=[];
teout=[];
xeout=[];
ieout=[];
```

```
while tstart<tfinal
    [t,x,te,xe,ie]=ode23(@f,[tstart,tfinal],x0,options);
    tstart=t(end);
    x0=[0;-0.9*xe(2)];
    tout=[tout;t];
    xout=[xout;x];
    teout=[teout;te];
    xeout=[xeout;xe];
    ieout=[ieout;ie];
end

plot(tout,xout(:,1),'b',teout,xeout(:,1),'bo')
title('Ball trajectory and events')
xlabel('t(sec)')
ylabel('height')
grid

function dx=f(t,x)
global g
dx=[x(2);-g];

function [value,isterminal,direction]=events(t,x)
value=x(1);
isterminal=1;
direction=-1;
```

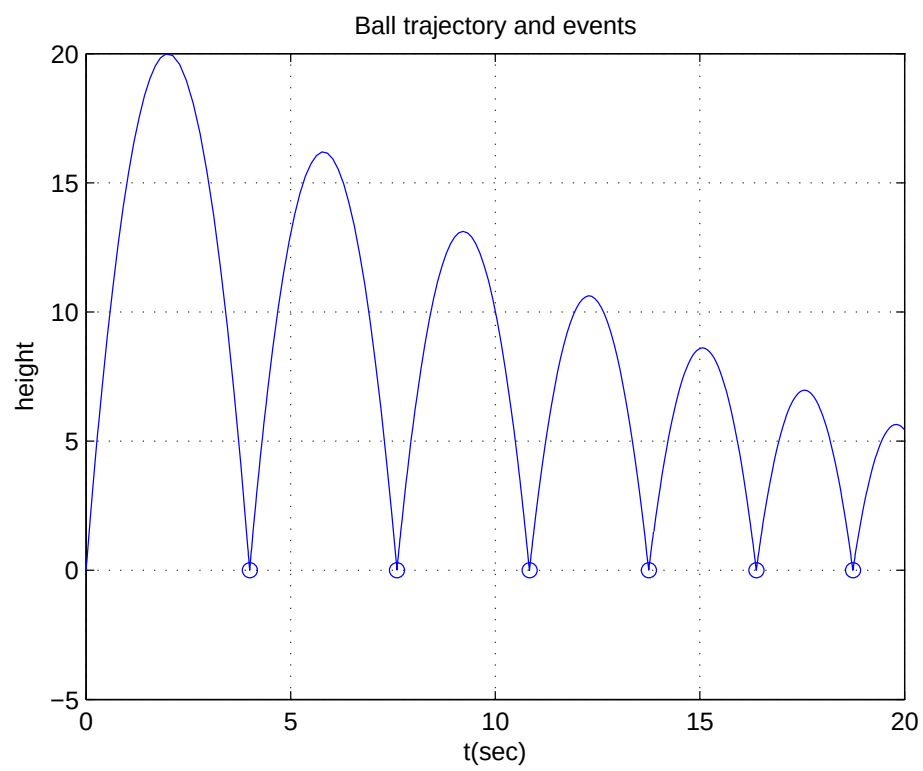


Figure 2.4: Πρόβλημα ανίχνευσης γεγονότος

ΒΙΒΛΙΟΓΡΑΦΙΑ

- [1] Adrian Biran & Moshe Breiner, MATLAB 5 για Μηχανικούς, Μετάφραση, Τζιόλας, Θεσσαλονίκη, 1999
- [2] Differential Equations, MATLAB User's Guide - Chapter 15, The MathWorks Inc., Natick